Problem A. Asignación Aleatoria de Tareas

Source file name:	Asignacion.c, Asignacion.cpp, Asignacion.java, Asignacion.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`1/2/2 (C++/Java/Python) second(s) / 64 megabytes`
Author(s):	`Gabriel Gutiérrez Tamayo - UTP Colombia`

Alice y Bob son estudiantes de un curso de programación, pero ambos son bastante perezosos. Para su mala suerte, les ha tocado trabajar juntos en el próximo proyecto. Siempre que han formado equipo, siguen una dinámica en la que, en todo momento, uno de ellos está realizando una tarea o ambos están descansando. Cada vez que finalizan una tarea, o concluye su tiempo de descanso mutuo, deciden que hacer a continuación de la siguiente manera:

- Ambos lanzan un dado de seis caras.
- Si los valores obtenidos son distintos, quién obtuviese el número más alto, realiza la siguiente tarea.
- Si los valores son iguales, ambos descansan durante los próximos x minutos.

Todos en el curso conocen la dinámica de trabajo de Alice y Bob cuando forman equipo, y ahora su profesor también lo sabe. Intrigado por cuánto tardarían en completar el próximo proyecto, decidió asignarlos como equipo para analizar su desempeño.

El proyecto consta de n tareas, todas de dificultad similar. Basándose en su conocimiento sobre los estudiantes, el profesor ha estimado el tiempo, en minutos, que Alice y Bob podrían tardar en completar una sola tarea.

Input

La primera línea contiene un entero q ($1 \leq q \leq 10^5$), que representa el número de consultas. Cada consulta se describe en una línea con cuatro enteros $n,\ a,\ b\ x$ ($1 \leq n, a, b, x \leq 10^6$), donde n es el número de tareas, a y b son los tiempos que tardan Alice y Bob en completar una tarea, respectivamente, y x es el tiempo de descanso mutuo.

Output

Para cada consulta, imprima en una línea un número indicando el tiempo esperado, en minutos, que tardarán en completar todas las tareas. Para cada consulta, su respuesta es considerada correcta si su error absoluto o relativo no excede 10^{-4}.

Example

Input	Output
4 1 10 20 5 4 10 20 5 3 10 13 10 5 30 40 20	16 64 40.5 195

Use fast I/O methods

Problem B. Borrar, borrar, borrar, ...

Source file name:	Borrar.c, Borrar.cpp, Borrar.java, Borrar.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/2/2 (C++/Java/Python) second(s) / 64 megabytes
Author(s):	Hugo Humberto Morales Peña - UTP Colombia

El profesor Humbertov Moralov ha tenido tiempo de sobra en sus últimas vacaciones y lo ha invertido en escribir retos de programación para evaluar a sus nuevos estudiantes del curso de programación.

El profesor Moralov ha dejado un mensaje oculto en un texto (entiéndase, en una cadena de caracteres). Para poder descubrir el mensaje oculto se tienen que eliminar del texto todas las ocurrencias de una lista de caracteres. Además, se sabe de antemano que el mensaje oculto no comienza ni finaliza con uno o más espacios en blanco y que no contiene el mismo símbolo de forma consecutiva.

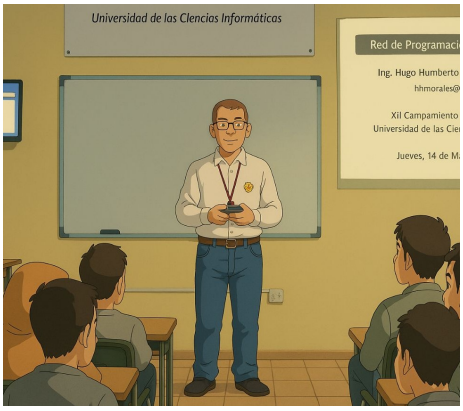


Imagen del profesor Humbertov Moralov generada con ChatGPT en modo "Studio Ghibli"

Input

La entrada contiene un único caso de prueba conformado por dos líneas.

La primera línea contiene un texto cuya longitud es de mínimo 10 y de máximo 10^6, el cual puede contener las letras en minúscula del alfabeto Inglés, los dígitos del 0 al 9 y el símbolo de espacio en blanco (ASCII 32).

La segunda línea contiene una cadena de caracteres (cadena de símbolos) que no pertenecen al mensaje encerrada entre el corchete que abre ("`[`") y el corchete que cierra ("`]`"), cuya longitud es de mínimo 1 y de máximo 36, la cual puede contener letras en minúscula del alfabeto Inglés, dígitos el 0 al 9, o inclusive el símbolo del espacio en blanco (ASCII 32).

Output

Imprimir en una sola línea la cadena de caracteres que representa el mensaje oculto. La línea debe finalizar con el símbolo de cambio de línea ("`\n`").

Nota: El mensaje oculto más corto es de una sola letra o un solo dígito. Además, el mensaje oculto no contiene espacios en blanco ni al comienzo ni al final.

Example

Input
king lokif bfbbddddafbbtggffaf kong bsbtbgrgugfcgtfugrfegg bigbig kjong kiko [bfghijklmno]
Output
data structure

Input
auub attb appb aoob appb aeeb annb a22b a00b a22b a55b [abc]
Output
utpopen2025

Use fast I/O methods

Problem C. Café

Source file name:	Cafe.c, Cafe.cpp, Cafe.java, Cafe.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`1/5/2 (C++/Java/Python) second(s) / 64 megabytes`
Author(s):	`Hugo Humberto Morales Peña - UTP Colombia`

El profesor Humbertov Moralov tiene la fortuna de trabajar en una de las mejores Universidades del Eje Cafetero en Colombia. ¡Obviamente su bebida favorita para comenzar el día es una buena taza de café!, pero ... tiene una obsesión con respecto a mezclar el café con agua, donde debe quedar un 50% de café y un 50% de agua. Si no se respeta esto entonces el profesor considerará que el café esta muy aguado o muy espeso.



Taza de café del profesor Humbertov Moralov

La taza de café del profesor Moralov tiene la forma de un cono truncado volteado, con radio de la base r (o radio del fondo de la taza), radio de la parte superior R (o radio de la boca de la taza) y altura h.

El grupo de fanes del profesor Moralov (los cuales se hacen llamar los **Moralovers**) se ha dado cuenta de la problemática que vive día a día su profesor con la taza de café y necesitan de tu ayuda para que por favor le indiques hasta donde debe de llenar de café su taza para que la combinación con el agua sea perfecta.

Los Moralovers te facilitan la siguiente información que podría llegar a ser de utilidad:

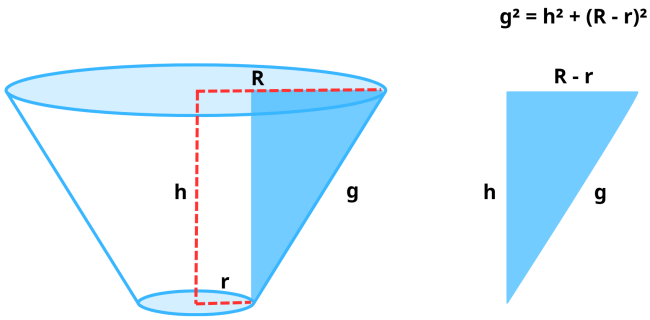


Figura 1. Cono Truncado

La fórmula para calcular el volumen del cono truncado es:

$$V = \frac{1}{3} \cdot \pi \cdot h \cdot \left(R^2 + r^2 + R \cdot r\right)$$

Input

La entrada comienza con un número entero t $\left(1 \leq t \leq 5 \cdot 10^4\right)$ el cual representa el total de casos de prueba. Cada uno de los casos de prueba consta de tres números reales r R h $\left(1.5 \leq r \leq 4.0,\ \ 2.5 \leq R \leq 6.0,\ 7.0 \leq h \leq 10.0,\ \ r \leq R\right)$, los cuales representan respectivamente el radio del fondo, el radio de la boca y la altura de la taza en centímetros.

Output

Por cada caso de prueba imprimir en una sola línea un número real que representa la altura hasta la cual el profesor Moralov debe de llenar su taza de café para obtener la combinación perfecta. La altura (en centímetros) será considerada correcta si el error absoluto o relativo no excede al 10^{-6}.

Example

Input	Output
4 2.5 3.8 8.8 2.0 4.0 10.0 1.5 2.5 9.0 3.0 3.0 8.8	5.271806416 6.509636245 5.561206129 4.400000000

Use fast I/O methods

Problem D. Domino a Ciegas

Source file name:	Domino.c, Domino.cpp, Domino.java, Domino.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`1/2/2 (C++/Java/Python) second(s) / 64 megabytes`
Author(s):	`Gabriel Gutiérrez Tamayo - UTP Colombia`

El dominó es un juego de mesa jugado por turnos que consta de 28 fichas diferentes, donde cada ficha está identificada por dos enteros entre 0 y 6. En esta versión del juego participan dos jugadores, y sus fichas permanecerán boca abajo; solo podrán ver una ficha en cada turno. Antes de iniciar una partida, cada jugador selecciona 7 fichas y las organiza en una fila de izquierda a derecha. En cada turno, un jugador voltea su ficha más a la izquierda, para poder ver su valor y, si alguno de sus dos valores coincide con alguno de los dos extremos de la secuencia de juego, puede juntarla al respectivo extremo por el lado de coincidencia. De lo contrario, coloca su ficha nuevamente al inicio, volteada boca abajo, pero esta vez de derecha a izquierda. Después de que un jugador mueve su ficha a uno de los extremos, o decide pasar, sigue el turno del otro jugador.

Una partida finaliza cuando un jugador se queda sin fichas o si desde el último turno en el que se colocó una ficha, todas las fichas restantes fueron volteadas al menos una vez. Cuando la partida finaliza, gana el jugador que se quedó sin fichas. Si ambos jugadores quedaron con fichas, gana quien tenga el menor puntaje.

El **puntaje** de un jugador al final de una partida es la suma de los puntos de las fichas que no usó.

Cada jugador lleva la cuenta de en cuántas diferentes fichas ha visto un número específico, contando dos veces para fichas dobles. A esto se le llama el contador del número. Cada número entero del 0 al 6 tiene un contador. Ambos jugadores son muy cuidadosos de no mostrar su ficha al otro jugador al momento de voltearla, por lo cual cada jugador lleva la cuenta con base en las fichas que él ha volteado o que ya han sido colocadas.

Alice y Bob quieren jugar unas partidas, y ellos siempre usan la siguiente estrategia cada vez que es su turno:

- Si ambos extremos son iguales y alguno de los valores de su ficha coincide con los extremos, junta la ficha a cualquiera de los dos extremos.
- Si los extremos son diferentes y la ficha coincide con uno solo, junta la ficha a ese extremo.
- Si los extremos son diferentes y la ficha coincide con ambos, la ficha se junta al extremo con el contador más bajo. En caso de empate, se junta al extremo de menor valor.

Dadas las fichas que tiene cada jugador, determine quién gana la partida. Alice siempre tiene el primer turno y siempre colocará su primera ficha en ese turno.

Input

La primera línea contiene un entero t ($1 \leq t \leq 10^4$), indicando el número de casos de prueba. Cada caso de prueba consta de dos líneas, cada una conteniendo 7 pares de enteros a_i, b_i ($0 \leq a_i, b_i \leq 6$), describiendo en la primera las fichas de Alice y, en la segunda, las fichas de Bob. Se garantiza que, para cada caso de prueba, las 14 fichas son distintas entre sí.

Output

Para cada caso de prueba, imprima una línea con el formato W T A B, donde W es una cadena indicando quién ganó la partida, `"Alice"`, `"Bob"` o `"Draw"` en caso de empate, T es el número de turnos que duró la partida, y A y B son los puntajes finales de Alice y Bob, respectivamente.

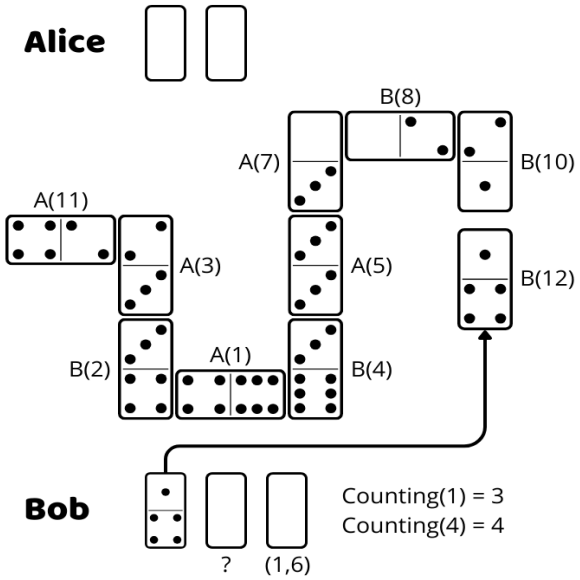


Figura 2. Primer caso de prueba, turno 12

Example

Input	Output
3 4 6 2 3 3 3 0 3 5 3 4 2 6 2 4 3 3 6 1 6 0 2 2 1 1 4 1 1 0 3 1 5 5 4 1 6 4 2 5 0 1 3 3 2 0 1 4 3 4 4 5 3 1 4 2 0 3 5 5 6 3 3 4 1 6 2 2 2 4 4 3 2 6 3 3 4 1 5 6 1 4 2 6 0	Bob 16 16 9 Alice 26 6 16 Draw 14 12 12

Use fast I/O methods

Problem E. Ernesto

Source file name:	Ernesto.c, Ernesto.cpp, Ernesto.java, Ernesto.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`2/3/3 (C++/Java/Python) second(s) / 256 megabytes`
Author(s):	`Santiago Gutiérrez Alzate - Google`

Ernesto es un empresario que tiene una gran cantidad de ideas para hacerse millonario, el problema es que no tiene tiempo para desarrollarlas (porque mantiene muy ocupado) y por ello necesita encontrar estudiantes que las desarrollen por él. Sin embargo, Ernesto tiene un par de restricciones:

1. Los estudiantes tienen que trabajar gratis para él, ya que es muy tacaño y no le gusta pagar.
2. Solamente tiene tiempo para "explotar" a un estudiante a la vez.



Globos de maratones de programación, Semillero In Silico, UTP

Como es natural, la oferta no parece muy atractiva para los estudiantes, así que lo que Ernesto hace es contratar a los estudiantes sin decirles cuanto les va a pagar, de esta manera ellos solo se dan cuenta de que están trabajando gratis cuando intentan reclamar su primer sueldo.

Como muchos estudiantes son amigos entre ellos, si Ernesto contrata a un estudiante, este estudiante, después de no recibir ningún sueldo, advertirá a todos sus amigos para que no trabajen para Ernesto. Los amigos de este estudiante, a su vez, advertirán a todos sus otros amigos, quienes advertirán a todos sus amigos que aun no conozcan a Ernesto, y así sucesivamente todo el grupo de amigos y amigos de amigos del estudiante explotado se enterara. Obviamente, llegara un momento en que todos los estudiantes serán avisados, tras lo cual Ernesto no podrá seguir contratando.

Ya que cada estudiante tiene una tolerancia a la explotación H (que determina el numero de horas que trabajaría sin preguntar por su sueldo), Ernesto te ha encomendado la tarea de que le digas a quienes debe contratar con el fin de maximizar la cantidad de horas que los estudiantes van a trabajar para él. Tienes suerte, esta vez el tacaño de Ernesto te ha prometido un globo si logras asesorarlo correctamente.

Input

La entrada comienza con un número entero positivo T $(1 \leq T \leq 10)$, el cual representa el total de casos de prueba. La primera línea de cada caso de prueba contiene dos números enteros N M $\left(1 \leq N \leq 10^5,\ 0 \leq M \leq 10^6\right)$, que representan respectivamente, el número de estudiantes y el número de relaciones de amistad. Las siguientes N líneas contienen una cadena de caracteres y un número entero positivo, S H $\left(1 \leq |S| \leq 10,\ 1 \leq H \leq 10^8\right)$, que representan respectivamente, el nombre del estudiante y su cantidad de horas de tolerancia a la explotación. El nombre del estudiante contiene únicamente símbolos en minúscula del alfabeto inglés. Para cada caso de prueba se garantiza que no hay dos estudiantes con el mismo nombre. Además, también se garantiza que no hay dos estudiantes con la misma cantidad de horas de tolerancia a la explotación.

El caso de prueba continúa con M líneas, cada una de ellas conteniendo un par de cadenas A B (lo cual es equivalente a B A, con $A \neq B$, $1 \leq |A|, |B| \leq 10$) que indica que los estudiantes con nombres A y B son amigos. Se garantiza que cada relación de amistad aparecerá a lo sumo una vez por cada caso de prueba.

Nota 1: La suma de los valores de N en los T casos de prueba es a lo sumo un 10^5.

Nota 2: La suma de los valores de M en los T casos de prueba es a lo sumo un 10^6.

Output

Para cada caso de la entrada se debe imprimir en una línea "`Case idCase:`", las comillas son solamente por claridad, donde `idCase` se reemplaza por el número del caso de prueba, seguido por tantas líneas como sean necesarias de los nombres de los estudiantes en orden alfabético. Estos son todos los estudiantes que Ernesto debe contratar con el fin de maximizar la cantidad de horas que los estudiantes de dicho caso de prueba van a trabajar gratis para él. Para mayor claridad mirar las entradas y salidas del ejemplo a continuación.

Example

Input	Output
2 7 4 arturo 100 daniel 15 carlos 50 carolina 20 jorge 1000 jessica 16 susana 1 daniel arturo carlos carolina carolina jessica susana jessica 5 4 sara 10 marcela 20 dario 25 adrian 26 laura 15 marcela sara dario marcela laura dario adrian laura	Case 1: arturo carlos jorge Case 2: adrian

Use fast I/O methods

Problem F. Felipe Felipe y la Estructura de Datos Cola

Source file name:	Felipe.c, Felipe.cpp, Felipe.java, Felipe.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`1/2/2 (C++/Java/Python) second(s) / 64 megabytes`
Author(s):	`Hugo Humberto Morales Peña - UTP Colombia`

Felipe Felipe inicialmente tiene una cola vacía y se le dan algunas consultas sobre ella. Las consultas son operaciones básicas que se pueden realizar sobre la estructura de datos *Cola*, tales como `Enqueue` (insertar un elemento al final de la cola), `Dequeue` (borrar el elemento que se encuentra al frente de la cola). También, es necesario conocer en cualquier momento el valor de la suma de todos los elementos que se encuentran actualmente en la cola. La tarea a realizar es procesar las consultas dadas.

Input

La primera línea contiene un número entero positivo T $(1 \leq T \leq 100)$ denotando el número de casos. Cada caso comienza con una línea que contiene un número entero positivo Q $\left(1 \leq Q \leq 10^4\right)$ denotando el número de consultas a procesar, seguidas por exactamente Q líneas basadas en el siguiente formato:

- 1 V: Operación `Enqueue` V $\left(0 \leq V \leq 10^4\right)$, inserta V al final de la cola.
- 2: Operación `Dequeue`, borra el elemento que se encuentra al frente de la cola. Si la cola está vacía entonces no se hace nada.
- 3: Imprimir en una línea el valor de la suma de todos los elementos que se encuentran actualmente en la cola. Si la cola está vacía, imprimir en una línea el mensaje: `Empty!`

Output

Para cada consulta del tipo 3 calcular e imprimir en una sola línea el valor de la suma de todos los elementos que se encuentran actualmente en la cola. Si la cola está vacía, imprimir en una línea el mensaje: `Empty!`

Example

Input	Output
2 12 2 1 10 1 15 3 2 2 3 1 18 3 1 10 1 1 3 3 3 1 999 3	25 Empty! 18 29 Empty! 999

Problem G. Guanex y las Colas de Prioridad

Source file name:	Guanex.c, Guanex.cpp, Guanex.java, Guanex.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`3/3/3 (C++/Java/Python) second(s) / 128 megabytes`
Author(s):	`Hugo Humberto Morales Peña - UTP Colombia`

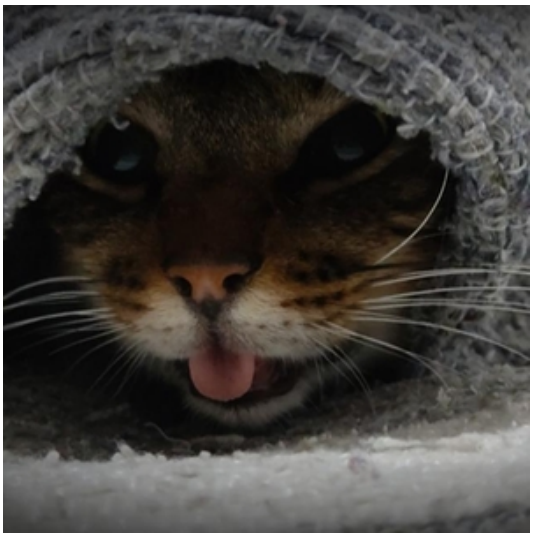


Foto de perfil del usuario Guanexxx en Codeforces

Guanex (Santiago Guaneme) es un joven de 14 años de Dosquebradas (Risaralda), pionero en el Eje Cafetero en las competencias de programación del IOI (International Olympiad in Informatic, en español, Olimpiada Internacional en Informática). Este joven que aún está en el colegio, fue quien obtuvo el mejor desempeño de la selección Colombia en el último mundial de programación del IOI en Egipto (IOI 2024 - Egypt).

Guanex se encuentra entrenando fuertemente para el IOI 2025 a realizarse en Sucre (Bolivia) del 27 de Julio al 3 de Agosto. Por este motivo se encuentra estudiando nuevas temáticas, una de ellas es las de colas de prioridad y te está invitando a que le ayudes a resolver el siguiente reto de programación.

Inicialmente se tiene una cola de prioridad y se dan algunas consultas sobre ella. Las consultas son operaciones básicas que se pueden realizar sobre la estructura de datos *Cola de Prioridad Mínima*, tales como `MinPQ_Insert` (insertar un elemento en la cola de prioridad mínima), `MinPQ_Extract` (extrae la primera ocurrencia del elemento mínimo en la cola de prioridad, siempre y cuando la cola de prioridad tenga elementos). También, es necesario conocer en cualquier momento el valor del elemento mínimo `MinPQ_Minimum` de los números enteros que se encuentran en la cola de prioridad. La tarea a realizar es procesar las consultas dadas.

Input

La entrada contiene un único caso de prueba. La primera línea contiene un número entero positivo Q $\left(1 \leq Q \leq 2 \cdot 10^6\right)$ denotando el número de consultas a procesar, seguidas por exactamente Q líneas basadas en el siguiente formato:

- 1 V: Operación `MinPQ_Insert` V $\left(0 \leq V \leq 10^6\right)$, inserta el número entero V en la cola de prioridad.
- 2: Operación `MinPQ_Extract`, si la cola de prioridad tiene elementos entonces extrae la primera ocurrencia del elemento mínimo. Si la cola de prioridad está vacía entonces no se hace nada.
- 3: Imprimir en una línea el valor del elemento mínimo de la cola de prioridad. Si la cola de prioridad está vacía, imprimir en una línea el mensaje: `Empty!`

Output

Para cada consulta del tipo 3 imprimir en una sola línea el valor del elemento mínimo de la cola de prioridad. Si la cola de prioridad está vacía, imprimir en una línea el mensaje: `Empty!`

Example

Input	Output
12 3 2 1 10 1 5 3 2 3 1 5 1 3 3 1 15 3	Empty! 5 10 3 3

Use fast I/O methods

Problem H. Humbertov y el Josephus de los No-Coprimos

Source file name:	Humbertov.c, Humbertov.cpp, Humbertov.java, Humbertov.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`1/1/2 (C++/Java/Python) second(s) / 64 megabytes`
Author(s):	`Gabriel Gutiérrez Tamayo - UTP Colombia`

El profesor Humbertov Moralov dicta el curso de Estructuras de Datos y, como están próximos al examen de listas enlazadas, propone exonerar a uno de los estudiantes a través de un juego en el que se podría aplicar este tema. El proceso del juego es el siguiente:

- En una bolsa hay n pedazos de papel, cada uno con un número entre 1 y n, donde cada número aparece exactamente una vez.
- Cada uno de los n estudiantes saca uno de los pedazos de papel de la bolsa, y el número que tenga escrito, identificará al estudiante en el juego.
- El estudiante con el número 1 es excluido del juego.
- Los estudiantes restantes se paran formando un círculo, todos mirando hacia el centro, de modo que, para cada estudiante i $(2 \leq i \leq n-1)$, tenga a mano izquierda al estudiante $i+1$, y el estudiante n tenga a mano izquierda al estudiante 2.
- Se realizan varias rondas hasta que solo quede un estudiante. En cada ronda, un estudiante escribe su número sobre una hoja y la rota a mano izquierda. Cada vez que un estudiante recibe la hoja, observa el número allí, calcula el MCD (Máximo Común Divisor) entre su número y el de la hoja, y si el resultado es mayor que 1, se retira del juego; de lo contrario, sigue rotando la hoja a mano izquierda. El estudiante que inicia en la primera ronda es el número 2, y en las rondas posteriores inicia el estudiante a mano izquierda del estudiante eliminado.

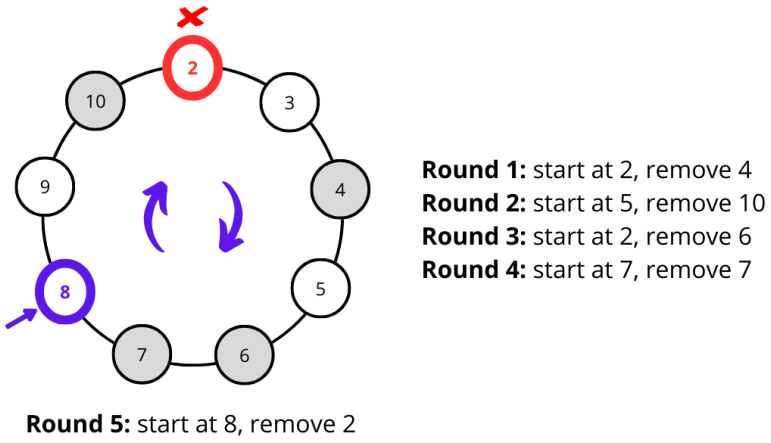


Figura 3. Primer caso de prueba, ronda 5

Dado el número de estudiantes, su tarea es determinar quién gana el juego.

Input

La primera línea contiene un entero t $(1 \leq t \leq 100)$, indicando el número de casos de prueba. Cada caso de prueba consiste en una línea con un entero n $\left(3 \leq n \leq 10^4\right)$, indicando el número de estudiantes.

Se garantiza que la suma de n en todos los casos de prueba es a lo sumo 10^4.

Output

Para cada caso de prueba, imprima una línea con un entero, indicando el número del estudiante ganador del juego.

Example

Input	Output
4 3 7 8 10	3 3 3 8

Problem I. Internal Triangles (Triángulos Internos)

Source file name:	Internal.c, Internal.cpp, Internal.java, Internal.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`1/1/1 (C++/Java/Python) second(s) / 64 megabytes`
Author(s):	`Gabriel Gutiérrez Tamayo - UTP Colombia`

Dado el número de lados de un polígono regular convexo, determine cuántos triángulos distintos pueden formarse seleccionando tres de sus vértices.

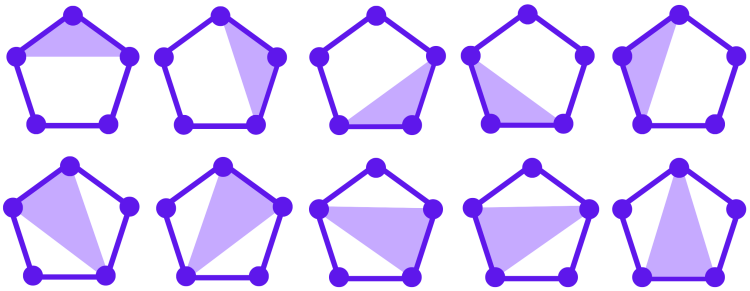


Figura 4. Todos los posibles triángulos del polígono convexo regular de 5 lados

Un polígono es **convexo** si todos sus ángulos internos son menores de 180 grados.

Un polígono es **regular** si todos sus lados y ángulos internos son iguales.

Dos triángulos se consideran distintos si al menos uno de los vértices de un triángulo no pertenece al otro.

Input

La primera línea contiene un entero q $\left(1 \leq q \leq 10^5\right)$, que indica el número de consultas. Cada una de las siguientes q líneas contiene un entero n $\left(3 \leq n \leq 10^{18}\right)$.

Output

Para cada consulta, imprimir una línea conteniendo la respuesta correspondiente. Dado que la respuesta a cada consulta puede ser muy grande, imprimirla módulo $10^9 + 7$.

Example

Input	Output
4 3 4 5 7	1 4 10 35

Use fast I/O methods

Problem J. Juego

Source file name:	Juego.c, Juego.cpp, Juego.java, Juego.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`3/3/3 (C++/Java/Python) second(s) / 128 megabytes`
Author(s):	`Hugo Humberto Morales & Gabriel Gutiérrez - UTP Colombia`

Emma y Otto son un par de niños muy talentosos que aprovechan su tiempo libre para jugar y practicar matemáticas al mismo tiempo. En estos momentos ellos se encuentran jugando "pares e impares" ... juego creado por ellos en el último mes.

El juego de "pares e impares" consiste de lo siguiente:

- En secreto, Emma escribe en una hoja de papel una lista con n números enteros no negativos.
- En secreto, Otto escribe en una hoja de papel una lista con n números enteros no negativos.
- En este punto los jugadores conocen las dos listas de números que se van a enfrentar y así será hasta el final del juego.
- Por cada turno:
  - Emma saca de su lista el número más grande y lo juega.
  - Otto saca de su lista el número más pequeño y lo juega.
  - Se suman los dos números, si el resultado es un número impar entonces este es adicionado a la lista de Otto, de lo contrario es dividido por 2 y el nuevo valor es adicionado a la lista de Emma.
- El juego termina cuando al final de un turno una de las dos listas queda vacía.
- El juego es perdido por quien queda con la lista vacía.

Emma y Otto necesitan de tu ayuda para simular el juego e indicar el turno en el que termina y el ganador del mismo, esto con el fin de validar los resultados a los cuales están llegando con el juego ... ojo ... ellos no solamente están jugando, también están practicando las funciones matemáticas de: mínimo, máximo, suma, división y residuo.

Input

La entrada comienza con un número entero t $\left(1 \leq t \leq 10\right)$ el cual representa el total de casos de prueba. Cada uno de los casos de prueba consta de tres líneas. La primera línea contiene un número entero n $\left(1 \leq n \leq 5 \cdot 10^5\right)$ el cual representa el tamaño de las listas. La segunda línea contiene n números enteros no negativos en el rango de $\left[0,\ 10^6\right]$ separados por espacio en blanco, representando la lista de números de Emma. La tercera línea contiene n números enteros no negativos en el rango de $\left[0,\ 10^6\right]$ separados por espacio en blanco, representando la lista de números de Otto.

Nota: Se garantiza que la suma de los valores de n en los t casos de prueba es a lo sumo un $5 \cdot 10^5$.

Output

Por cada caso de prueba imprimir en una sola línea un número entero que representa la cantidad de turnos del juego hasta que termina y el nombre del jugador ganador, ya sea `Emma` u `Otto`.

Nota: En este juego nunca hay empates, siempre hay un ganador.

Example

Input	Output
5 5 1 1 1 1 1 2 2 2 2 2 5 5 4 3 2 1 5 1 3 1 5 4 2 4 4 2 6 2 8 4 5 13 17 2 0 4 5 1 11 3 18 5 5 1 11 3 18 13 17 2 0 4	5 Otto 9 Emma 5 Emma 8 Otto 8 Emma

Use fast I/O methods

Problem K. K^2

Source file name:	Kcuadrado.c, Kcuadrado.cpp, Kcuadrado.java, Kcuadrado.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`1/2/2 (C++/Java/Python) second(s) / 64 megabytes`
Author(s):	`Gabriel Gutiérrez Tamayo - UTP Colombia`

Dado un triángulo equilátero de lado k, calcular el número máximo de triángulos equiláteros de lado 1, que se pueden colocar dentro de él, sin superponerse.

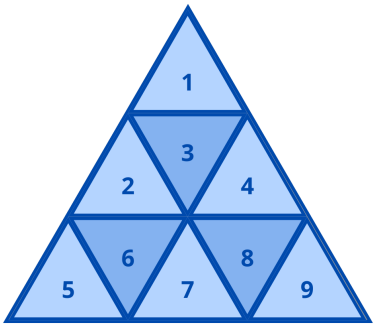


Figura 5. Solución óptima para un triángulo equilátero con lados de longitud 3

Un **triángulo equilátero** es un triángulo en el que todos sus lados tienen la misma longitud.

Input

La primera línea contiene un entero q ($1 \leq q \leq 5 \cdot 10^5$), que indica el número de consultas. Cada una de las siguientes q líneas contiene un entero k ($1 \leq k \leq 10^{18}$).

Output

Para cada consulta, imprima una línea conteniendo la respuesta correspondiente. Dado que la respuesta a cada consulta puede ser muy grande, imprimirla módulo $10^{18} + 3$.

Example

Input	Output
3 1 2 3	1 4 9

Use fast I/O methods

Problem L. Legend

Source file name:	Legend.c, Legend.cpp, Legend.java, Legend.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`1/3/3 (C++/Java/Python) second(s) / 128 megabytes`
Author(s):	`Hugo Humberto Morales Peña - UTP Colombia`

UTP - Legend ha sido el mejor equipo que ha tenido la Universidad Tecnológica de Pereira en el ICPC (International Collegiate Programming Contest, en español, Competición Internacional Universitaria de Programación).



Figura 6. Equipo UTP - Legend. Subcampeón Maratón Nacional Colombiana 2012. En la foto de izquierda a derecha: Diego Alejandro Agudelo España, Sebastián Gómez González y Santiago Gutiérrez Alzate.

En una sola foto se encuentran:

- Tres egresados del programa de Ingeniería de Sistemas y Computación de UTP
- Uno de los fundadores del Semillero de Investigación In Silico (Sebas)
- El mejor ECAES de Ingeniería de Sistemas de Colombia en el año 2010 (Sebas)
- Dos ganadores de la Beca Jorge Roa Martínez (Sebas y Diego)
- Dos egresados de la maestría de Ingeniería de Sistemas y Computación de UTP (Sebas y Diego)
- Un doctor en Machine Learning (Sebas)
- Un candidato a doctor en Machine Learning (Diego)
- Dos Ingenieros de Desarrollo en Google (Santi en Estados Unidos y Sebas en Alemania), entre los dos suman más de 15 años trabajando en dicha empresa
- Un doble pasante como estudiante en Google (Estados Unidos e Inglaterra - Diego)
- Cinco idiomas (español, inglés, portugués, alemán y francés)

- El primer equipo de UTP en alcanzar la clasificación a un mundial de programación (ICPC World Finals 2013 - San Petersburgo, Rusia)
- El **primer y único** equipo Colombiano en alcanzar el pódium en el **ICPC Challenge** (ICPC World Finals 2013 - San Petersburgo, Rusia)
- Los culpables de vincular (desde el año 2011 hasta la fecha) al autor de este reto al proceso de maratones de programación
- ¡Tres pioneros que nos enseñaron con el ejemplo que la Universidad Tecnológica de Pereira no tiene nada que envidiarle a ninguna otra Universidad colombiana!

El equipo **UTP - Legend** es nuestro referente a seguir, por ese motivo, en honor a ellos va este reto de programación.

Un sábado en la tarde, después de participar en una de las tantas competencias que realizar la Red de Programación Competitiva (RPC), los tres integrantes del equipo UTP - Legend quieren caminar desde UTP (sede presencial de la RPC en el Eje Cafetero) a sus respectivas casas en un recorrido óptimo, pero ... Sebas, Santi y Diego quieren caminar la mayor distancia posible juntos para seguir debatiendo sobre los retos que trabajaron en la competencia.

Nuestro equipo de leyendas se encuentra con múltiples ocupaciones y necesita de tu ayuda para que les indiques cuanta es la distancia que pueden caminar juntos, y las respectivas distancias que cada uno debe caminar después, obviamente en un recorrido óptimo.

Input

La entrada del problema consiste de un solo caso de prueba. El caso de prueba es descrito en varias líneas. La primera línea del caso de prueba consiste de seis números enteros N, U, S_e, S_a, D y C separados por un solo espacio en blanco. El valor N ($4 \leq N \leq 10^5$) es el número de intersecciones de calles en la ciudad, cada intersección de calles es identificada por un número entero entre 1 y N. Los valores U, S_e, S_a y D ($1 \leq U, S_e, S_a, D \leq N$) son los identificadores de las intersecciones de las calles donde están localizados respectivamente la Universidad, la casa de Sebastián, la casa de Santiago y la casa de Diego, estos cuatro identificadores de intersecciones de calles son diferentes. C ($3 \leq C \leq 10^5$) es el número de calles de la ciudad. Cada una de las siguientes C líneas contiene la descripción de una calle. Cada calle es descrita usando dos números enteros E_1 y E_2 ($1 \leq E_1, E_2 \leq N,\ E_1 \neq E_2$) separados por un solo espacio en blanco, donde E_1 y E_2 identifican dos intersecciones de calle que son los puntos de finalización de la calle. Usted puede asumir que existen rutas desde la intersección de calle U hasta las intersecciones de calles S_e, S_a y D.

Output

La salida del caso de prueba consiste de una sola línea que debe contener cuatro números enteros R_j, R_{se}, R_{sa} y R_d separados por un solo espacio en blanco, donde R_j es el máximo número de calles que Sebas, Santi y Diego pueden caminar juntos, R_{se} es el número total de calles que le faltan por caminar a Sebas, R_{sa} es el número total de calles que le faltan por caminar a Santi y R_d es el número total de calles que le faltan por caminar a Diego, para que cada uno de ellos llegue a su respectiva casa.

Example

Input	Output
6 1 5 6 4 7 1 2 2 3 3 4 3 5 3 6 4 6 5 6	2 1 1 1
6 1 5 6 4 9 1 2 2 3 2 5 3 4 3 5 3 6 4 5 4 6 5 6	2 0 1 1
6 3 1 5 4 9 1 2 2 3 2 5 3 4 3 5 3 6 4 5 4 6 5 6	0 2 1 1

Problem M. MST

Source file name:	MST.c, MST.cpp, MST.java, MST.py
Input:	`Standard`
Output:	`Standard`
Time / Memory limit:	`2/2/2 (C++/Java/Python) second(s) / 128 megabytes`
Author(s):	`Hugo Humberto Morales & Gabriel Gutiérrez - UTP Colombia`

En un primer curso de Estructuras de Datos en un programa de Ciencias de la Computación normalmente se comienza el trabajo sobre grafos y se presenta el tema de Árboles de Abarcamiento Minimal (AAM, en inglés, Minimum Spanning Tree - MST), los cuales pueden ser calculados con alguno de los algoritmos de Prim o Kruskal.

Sea el **Número AAM** la cantidad de árboles de abarcamiento minimal que se pueden calcular y extraer de un grafo hasta que este queda no conexo. Es decir, el Número AAM es la cantidad de veces que se puede repetir el proceso de generar y luego borrar un árbol de abarcamiento minimal sobre un grafo hasta que el grafo quede no conexo.

El trabajo a realizar consiste en calcular el Número AAM y el peso de cada uno de los árboles de abarcamiento minimal que se pueden generar a partir de un grafo no dirigido con pesos en las aristas.

Input

La entrada contiene un único caso de prueba conformado de varias líneas. La primera línea del caso de prueba consta de dos números enteros positivos N M $\left(7 \leq N \leq 10^4, 0 \leq M \leq \min\left(\frac{N \cdot (N-1)}{2},\ 10^5\right)\right)$, que representan el número de vértices y la cantidad de aristas, respectivamente. Cada una de las siguientes M líneas representan las aristas del grafo. Cada arista constará de tres números enteros positivos V_1 V_2 P $\left(1 \leq V_1, V_2 \leq N,\ V_1 \neq V_2,\ 1 \leq P \leq 10^8\right)$, donde V_1 y V_2 son los números de los vértices y P es el peso de la arista.

Nota 1: Se garantiza que no hay dos aristas diferentes con el mismo peso.

Output

La primera línea en la salida debe contener un número entero no negativo R el cual representa el Número AAM del grafo, seguido de R líneas cada una representando el peso del árbol de abarcamiento minimal que se va generando y extrayendo del grafo en el proceso.

Nota 2: Se garantiza que para los casos de prueba el reto de programación tiene solución única.

Nota 3: Se garantiza que como mínimo hay un vértice con grado de incidencia de máximo 50.

Example

Input	Output
7 21 1 2 30 1 3 15 1 4 10 1 5 77 1 6 63 1 7 61 2 3 83 2 4 25 2 5 60 2 6 65 2 7 40 3 4 41 3 5 71 3 6 20 3 7 86 4 5 75 4 6 45 4 7 35 5 6 82 5 7 21 6 7 31	3 122 251 412

Use fast I/O methods